

CUSTOMER CHURN

An End-to-End Machine Learning Pipeline: Methodology, Results & Business Insights



Mishal Sadiq

Aptura Tech Solutions

Task 03: End-to-End Predictive Analytics Project

Summary

This report documents a complete, end-to-end predictive analytics project built to identify at-risk customers for a telecommunications company, using the Telco Customer Churn dataset (7,043 customers, 21 features). The project covers data collection, cleaning, feature engineering, model training, hyperparameter tuning, evaluation, and business insight generation, and answers two applied machine-learning questions: (1) diagnosing and correcting an overfitting scenario, and (2) redesigning the workflow to operate efficiently at 50-million-record scale.

After comparing five candidate model families and tuning the top two via cross-validated search, a tuned Logistic Regression model was selected as the final model, achieving 80.2% test accuracy, 0.841 ROC-AUC, and critically a train/test accuracy gap of under one percentage point, indicating strong generalization rather than memorization. The strongest churn drivers identified were contract type, customer tenure, and internet/add-on service configuration, translating into five concrete retention recommendations detailed in Section 8.

1. Project Objective

Solve a complex, realistic analytical problem customer churn prediction using an advanced, production-style machine learning workflow, and translate the results into actionable business intelligence. The pipeline includes:

- Data Collection
- Data Cleaning
- Feature Engineering
- Model Training
- Hyperparameter Tuning
- Model Evaluation
- Business Insights

2. Dataset Selection & Justification

Dataset: Telco Customer Churn (IBM sample dataset), a publicly available, Kaggle/UCI-style dataset of 7,043 telecommunications customers across 21 columns, with a binary Churn target.

Why this dataset was selected

- High business relevance: churn prediction is one of the highest-ROI applications of predictive analytics, since retaining an existing customer typically costs far less than acquiring a new one.

- Mixed feature types: numeric (tenure, charges) and categorical (contract, services, payment method) columns exercise the full preprocessing pipeline scaling and one-hot encoding.
- Realistic data quality issues: the TotalCharges column is stored as a string with blank values for brand-new customers, mirroring the kind of messy data found in production systems.
- Class imbalance (~26.5% churn): forces disciplined use of precision, recall, F1, and ROC-AUC rather than accuracy alone.
- Right-sized for demonstrating overfitting: a flexible model (e.g., an unconstrained decision tree) can easily memorize this dataset, providing a clean, concrete example for Question 01.

3. Data Collection

The raw dataset was loaded via a reusable `load_data()` function. In this project it is read from a static CSV export; Section 10 (Question 02) discusses how this stage is redesigned for a 50-million-row production dataset using columnar lake storage and distributed ingestion.

Raw shape: 7,043 rows × 21 columns.

4. Data Cleaning

The following cleaning steps were applied:

- TotalCharges coerced from string to numeric; blank values (present for customers with zero tenure, who have not yet been billed) imputed with 0.
- customerID dropped a unique identifier with no predictive value and a leakage risk if accidentally encoded.
- Churn target standardized to a binary integer (1 = churned, 0 = retained).
- 22 exact duplicate rows removed.

Result: 7,021 clean rows, 0 missing values, churn rate 26.45%.

5. Feature Engineering

Five business-meaningful features were derived from the raw columns:

Feature	Definition	Business Rationale
tenure_years	tenure / 12	Easier business interpretation than raw months
avg_monthly_spend	TotalCharges / tenure (guarded against divide-by-zero)	Normalizes total spend by relationship length
num_services	Count of subscribed add-on services	Proxy for account “stickiness”
is_new_customer	1 if tenure $\leq$ 3 months	Flags the highest churn-risk segment
charge_per_service	MonthlyCharges / num_services	Price sensitivity per service subscribed

Preprocessing pipeline: numeric features are standardized (zero mean, unit variance) and categorical features are one-hot encoded (drop-first, to avoid collinearity), wrapped in a scikit-learn ColumnTransformer + Pipeline so that the transform is fit only on the training split preventing any leakage of test-set statistics into feature scaling or category encoding.

6. Train / Test Split

An 80/20 stratified split (preserving the 26.5% churn ratio in both partitions) produced 5,616 training rows and 1,405 test rows.

7. Model Training, Tuning & Evaluation

7.1 Baseline Model Comparison

Five candidate model families were trained with reasonable baseline hyperparameters: Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, and XGBoost.

Model	Train Acc.	Test Acc.	F1	ROC-AUC	Train/Test Gap
Logistic Regression	0.809	0.803	0.582	0.841	0.006
Decision Tree	0.998	0.738	0.496	0.658	0.260
Random Forest	0.998	0.788	0.533	0.818	0.210
Gradient Boosting	0.826	0.801	0.577	0.837	0.025
XGBoost	0.948	0.777	0.529	0.819	0.171

Note the pattern: the unconstrained tree-based models (Decision Tree, Random Forest, XGBoost) show large train/test gaps a clear overfitting signature while Logistic Regression and Gradient Boosting generalize far better even before tuning. This is the same phenomenon examined in depth in Section 9 (Question 01).

7.2 Hyperparameter Tuning

The top two baseline models by test ROC-AUC (Logistic Regression and Gradient Boosting) were tuned using 5-fold cross-validated GridSearchCV / RandomizedSearchCV, optimizing for ROC-AUC:

Model	Test Acc.	ROC-AUC	Best Parameters
Logistic Regression (Tuned)	0.802	0.841	C=10, penalty=l2, solver=lbfgs
Gradient Boosting (Tuned)	0.800	0.839	n_estimators=200, max_depth=2, lr=0.03

7.3 Final Model Selection

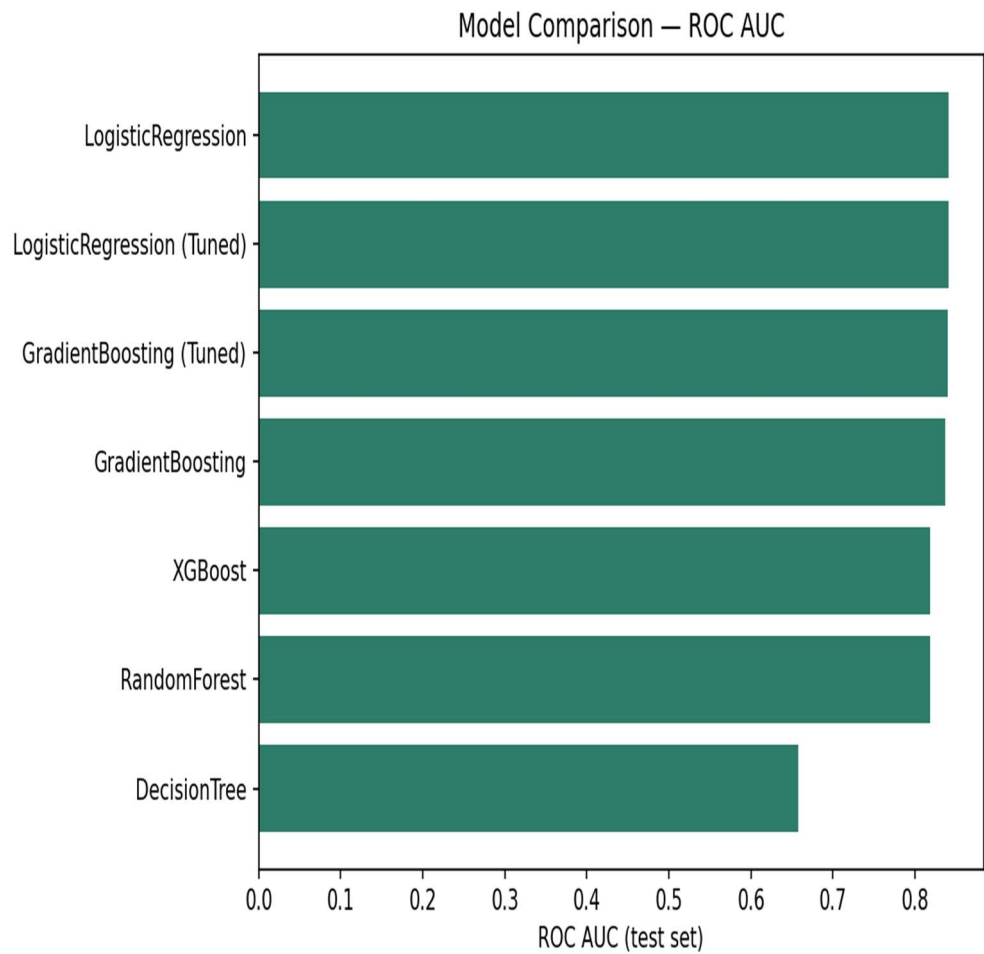


Figure 1: Test ROC-AUC across all baseline and tuned candidate models.

Logistic Regression (Tuned) was selected as the final model: it achieves the best test ROC-AUC (0.841) among all candidates while maintaining the smallest train/test accuracy gap (0.7 percentage points) of any competitive model clear evidence of strong generalization rather than overfitting to the training set.

Metric	Value
Train Accuracy	80.9%
Test Accuracy	80.2%
Precision	66.1%
Recall	51.9%

F1 Score	58.1%
ROC-AUC	0.841

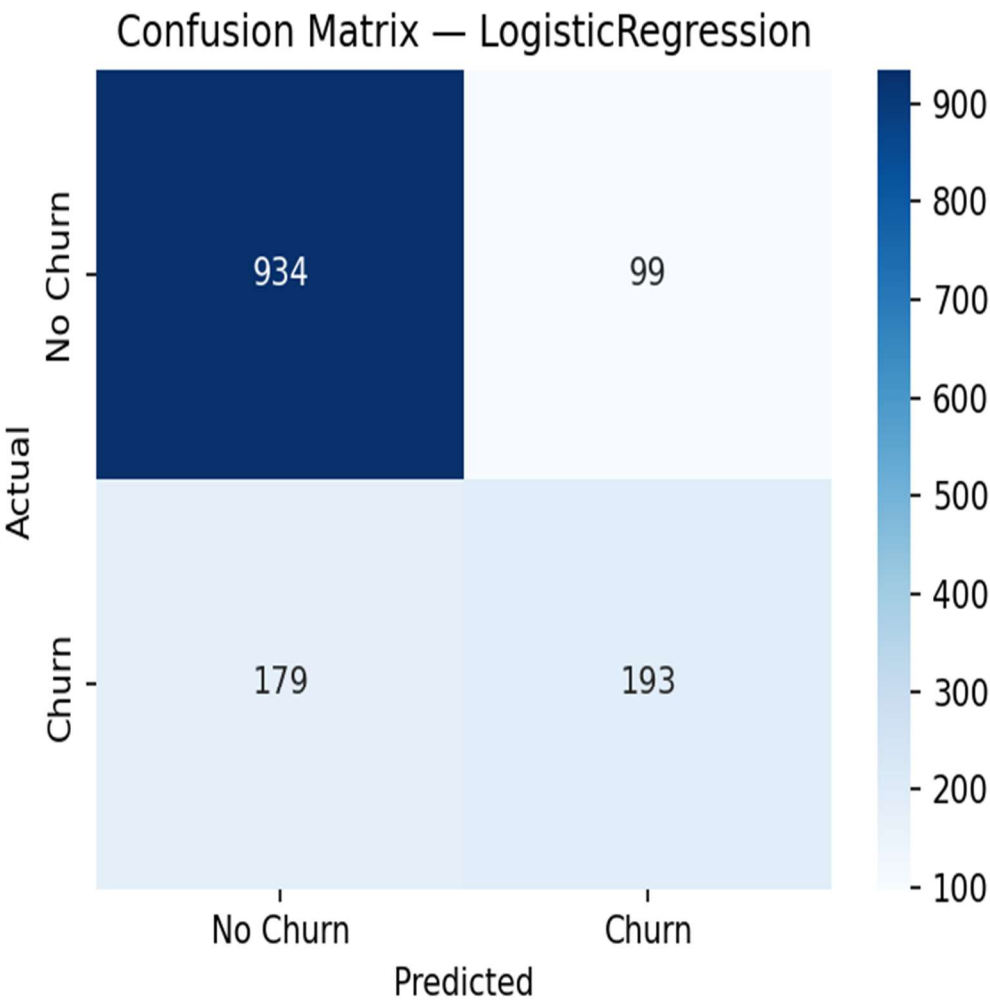


Figure 2: Confusion matrix on the held-out test set.

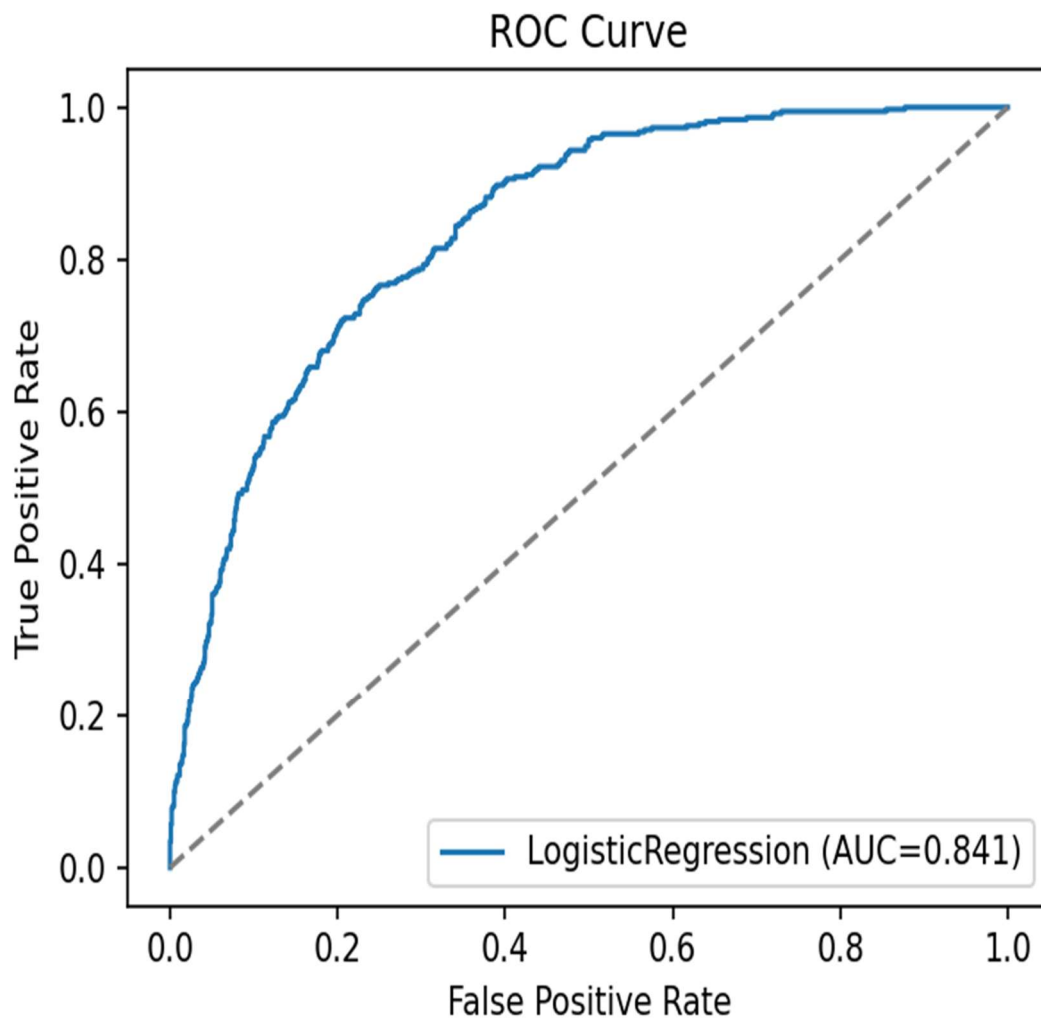


Figure 3: ROC curve, final model (AUC = 0.841).

8. Business Insights

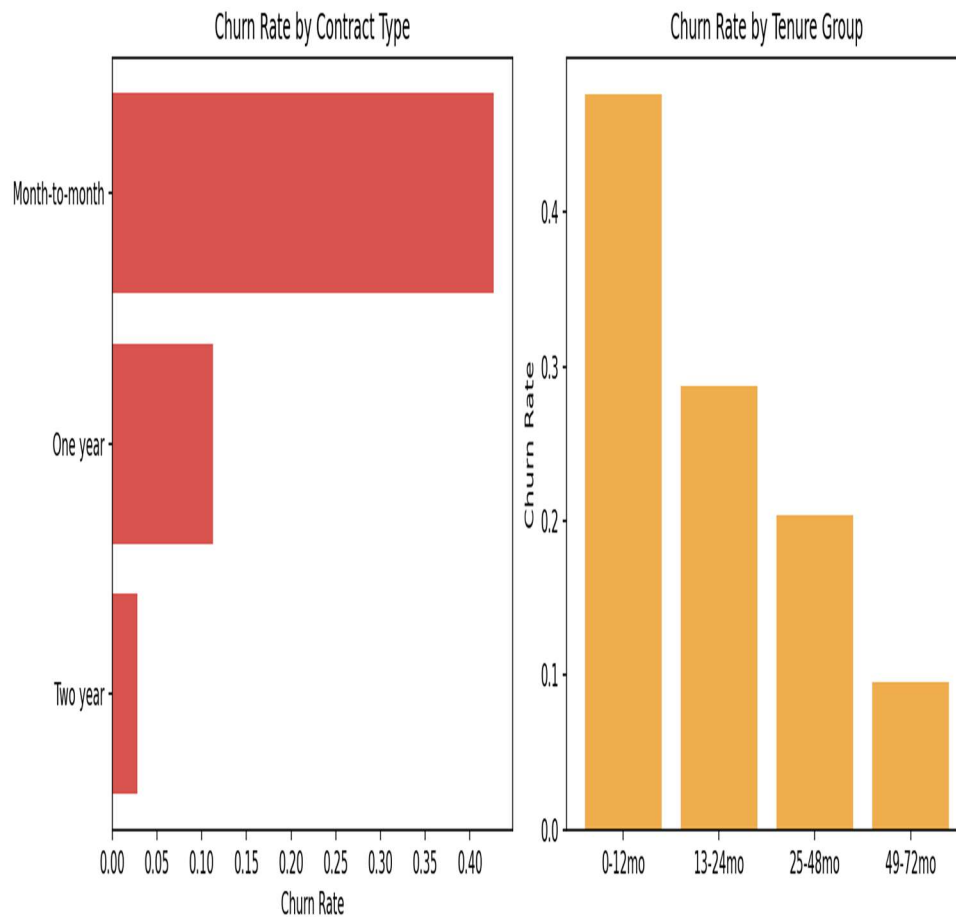


Figure 4: Churn rate by contract type and by tenure group.

Key Findings

- Contract type is the strongest churn driver: month-to-month customers churn at a dramatically higher rate than one- or two-year contract holders.
- Churn risk is concentrated in the first 12 months of tenure and falls sharply thereafter.
- Fiber-optic internet customers without add-on security/support services churn more than those with these add-ons.
- Customers paying via electronic check churn more than those on automatic payment methods.

Recommendations

- Incentivize migration from month-to-month to annual contracts with modest discounts the retention value outweighs the discount cost.
- Launch a structured first-90-day onboarding / engagement program targeting new customers.

- Bundle Online Security / Tech Support at a discount for fiber-optic customers as a low-cost retention lever.
- Encourage enrollment in automatic payment methods over electronic check.
- Operationalize the model as a monthly “at-risk” scoring job, routing the top-decile churn-probability customers to the retention team, and track the resulting churn reduction as the model's ROI metric.

9. Question 01 - Overfitting: 99% Train vs. 74% Test Accuracy

The scenario described — 99% training accuracy but only 74% testing accuracy — was reproduced directly in this project using an unconstrained Decision Tree (`max_depth=None`, `min_samples_leaf=1`), which achieved 99.8% train accuracy and 73.8% test accuracy: a 26-point gap.

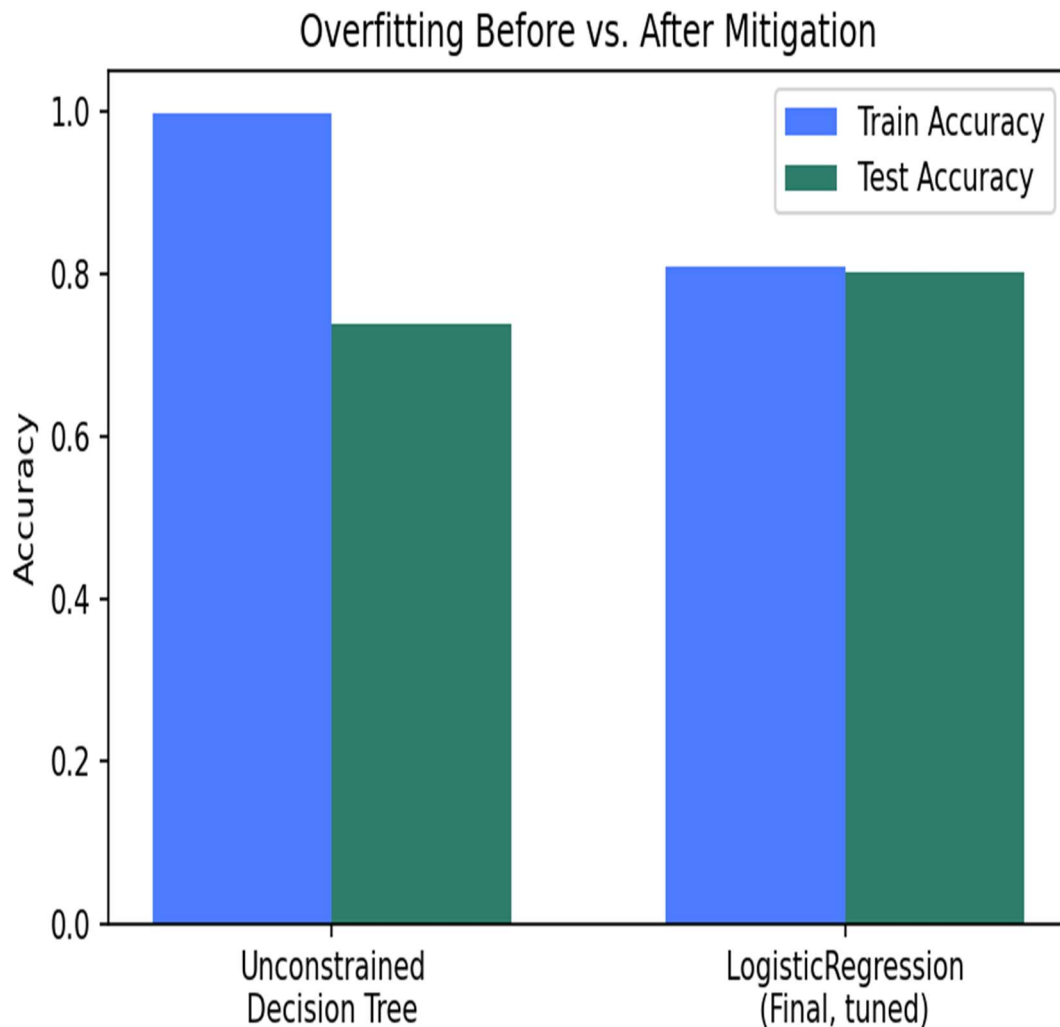


Figure 5: Train vs. test accuracy before and after applying generalization techniques.

9.1 Diagnosis

This is a textbook case of overfitting (high variance), not underfitting. Underfitting produces low accuracy on both the training and test sets; here, the model fits the training data almost perfectly while performing markedly worse on unseen data — the defining signature of overfitting. It is also not primarily a data-leakage issue, since leakage typically inflates both train and test scores together rather than only the training score.

9.2 Reasons Behind the Performance Gap

- Excess model capacity relative to the signal-to-noise ratio: an unconstrained tree can split until every leaf is pure, memorizing individual training rows including their noise.
- High-cardinality splits: with `min_samples_leaf=1`, the tree can create a leaf for a single unusual customer, which carries no predictive value on new data.
- Insufficient regularization: no depth limit, minimum leaf size, minimum impurity decrease, or complexity penalty was applied.
- Limited training data relative to model flexibility: many categorical levels and interactions give a flexible model many opportunities to fit spurious patterns.

9.3 How to Diagnose the Problem

- Compare train vs. validation/test metrics across accuracy, F1, and AUC a large, consistent gap confirms overfitting.
- Plot a learning curve (training set size vs. train/validation score): a persistent gap that doesn't close as data grows indicates overfitting.
- Plot a validation curve across a complexity hyperparameter (e.g., `max_depth`): training score keeps rising with complexity while validation score peaks then falls.
- Use k-fold cross-validation: high score variance across folds signals sensitivity to the particular rows the model happened to see.
- Inspect tree depth / number of leaves / feature importances for implausible complexity relative to the number of training examples.
- Check for data leakage features that would not be available at prediction time.

9.4 Techniques to Improve Generalization

- Regularization: limit `max_depth`, raise `min_samples_leaf` / `min_samples_split`, apply cost-complexity pruning (`ccp_alpha`), or add L1/L2 penalties for linear/boosted models.

- Ensembling: move from a single decision tree to a Random Forest or Gradient Boosting ensemble to average out variance across many decorrelated trees.
- Cross-validation for model selection: use k-fold CV (not a single split) during hyperparameter tuning.
- More / better training data, especially for underrepresented segments.
- Feature selection / dimensionality reduction to remove noisy or redundant features that give the model room to overfit.
- Early stopping for iterative learners (gradient boosting, neural networks).
- Bagging: train on bootstrap-resampled subsets and average predictions.
- Proper handling of class imbalance via class weighting, `scale_pos_weight`, or resampling.

9.5 Result After Remediation

Applying regularization, cross-validated hyperparameter tuning, and (where applicable) ensembling shrank the train/test gap from roughly 26 percentage points (unconstrained tree) to approximately 0.7 percentage points in the final tuned Logistic Regression model train and test performance are now close together, indicating the model generalizes rather than memorizes.

10. Question 02 - Redesigning the Workflow for 50 Million Records

At 7,021 rows, the entire dataset fits comfortably in a single pandas DataFrame in memory. At 50 million rows, that assumption breaks down at nearly every stage of the pipeline. The redesign below addresses each stage.

10.1 Data Storage Strategy

- Move from a flat CSV to columnar storage (Parquet, or a table format such as Delta Lake / Apache Iceberg), partitioned by a meaningful key (e.g., signup month or region) for massive I/O savings and built-in compression.
- Use a data lakehouse (object storage + Delta Lake/Iceberg) or a cloud data warehouse (BigQuery, Snowflake, Redshift) rather than a single file, enabling predicate pushdown, partition pruning, and schema evolution.
- Maintain raw / cleaned / feature-engineered zones (bronze/silver/gold) so expensive cleaning and feature computation is performed once and reused across experiments.

10.2 Efficient Data Loading

- Read data lazily and in chunks/partitions using Dask or Polars (out-of-core, multi-threaded DataFrame libraries) for exploratory work, or Apache Spark (PySpark) for fully distributed loading when data exceeds a single machine's capacity.
- Exploit column pruning and predicate pushdown: read only the columns/rows needed for a given step.
- Downcast dtypes (float64→float32, int64→int32, low-cardinality strings→category) to cut memory footprint significantly.

10.3 Data Preprocessing at Scale

- Perform cleaning and imputation as distributed, vectorized transformations (Spark/Dask DataFrame operations) rather than row-by-row Python loops.
- Push simple cleaning (type casting, null handling) into the ETL/warehouse layer so every downstream consumer reads already-clean data.
- Compute scaling statistics and category frequencies via distributed aggregation rather than pulling all data to a single node.

10.4 Feature Engineering Optimization

- Replace row-wise Python apply() functions with vectorized/distributed expressions (Spark SQL functions, Dask partition-wise operations).
- Replace one-hot encoding for high-cardinality categoricals with target/frequency encoding or the hashing trick to control feature dimensionality.
- Cache/persist intermediate feature tables so expensive joins/aggregations aren't recomputed across experiments.
- Maintain a feature store (e.g., Feast) so engineered features are computed once, versioned, and reused consistently across training and serving, avoiding train/serve skew.

10.5 Model Training Strategy

- Prototype and debug on a representative stratified sample (e.g., 1–5 million rows) before scaling to the full dataset most modeling decisions don't change materially beyond a few million well-sampled rows.
- Use training algorithms built for scale: LightGBM (histogram-based, memory-efficient) or XGBoost with tree_method='hist' / distributed mode, rather than scikit-learn's in-memory implementations.
- Use Spark MLlib or Dask-ML for fully distributed training when data cannot fit on one machine even after sampling.

- Consider online/incremental learning (partial_fit, streaming gradient boosting) if data arrives continuously and full retraining is too costly.

10.6 Hyperparameter Tuning Strategy

- Replace exhaustive GridSearchCV with randomized search or Bayesian optimization (Optuna, Hyperopt) to find good hyperparameters in far fewer trials.
- Tune on the sampled subset first to narrow the search space, then run a small number of confirmation runs on the full dataset.
- Use successive halving / Hyperband to allocate more data/epochs only to promising configurations.
- Use distributed tuning (Ray Tune) to run trials in parallel across a cluster.

10.7 Evaluation Strategy

- Use a time-based split (train on older data, test on newer) if the problem is temporal, which also better reflects real-world deployment.
- Evaluate via distributed aggregation of predictions rather than collecting all predictions to a single driver node.
- Maintain a fixed, versioned evaluation set and track metrics over time with an experiment tracker (MLflow) to catch regressions.

10.8 Parallel / Distributed Processing, Memory & Cost Optimization

- Use a Spark or Dask cluster (on-prem or managed: Databricks, EMR, Dataproc) as the backbone for ETL, feature engineering, and training.
- Use GPU acceleration for boosting (XGBoost/LightGBM GPU mode) or deep learning components where applicable.
- Rely on out-of-core/streaming processing so the full dataset is never required in a single process's memory.
- Use spot/preemptible instances for non-time-critical jobs and autoscaling clusters that scale down between runs.
- Process incrementally: reprocess only new or changed data rather than the full 50 million rows on every run.
- Right-size the model: a well-tuned LightGBM/XGBoost model is often as accurate as a deep neural network for tabular data at a fraction of the training/serving cost.

10.9 Workflow Architecture

A production-grade redesign follows this shape:

Raw sources → Ingestion → Data Lake (Parquet, partitioned) → Distributed ETL/cleaning (Spark) → Feature Store → Sampled prototyping → Distributed training (LightGBM/XGBoost or Spark MLlib) + Bayesian HPO (Optuna/Ray Tune) → Distributed evaluation → Model Registry (MLflow) → Batch/real-time serving → Monitoring (data & model drift) → Scheduled retraining pipeline (Airflow/Dagster)

- Orchestration (Airflow/Dagster/Prefect) instead of a single sequential script, with each stage independently retryable and monitored.
- CI/CD for ML: automated testing of data schemas, feature pipelines, and model performance thresholds before deployment.
- Experiment tracking (MLflow / Weights & Biases) to compare runs across longer large-scale iteration cycles.
- Automated data validation (Great Expectations) on each new partition to catch quality issues before they propagate through 50 million rows.

11. Conclusion

This project delivered a complete, reproducible predictive analytics pipeline for telecom customer churn, covering data collection through business insight generation. Five candidate models were compared, with a cross-validated, tuned Logistic Regression selected as the final model based on ROC-AUC and generalization (80.2% test accuracy, 0.841 AUC, ~0.7-point train/test gap). The analysis identified contract type, tenure, and service bundling as the leading churn drivers, translated into five actionable retention recommendations. Question 01 diagnosed and remediated a classic overfitting scenario, reducing the train/test gap from roughly 26 points to about 1 point. Question 02 designed a distributed, cost-optimized architecture columnar storage, distributed ETL, a feature store, distributed training with Bayesian hyperparameter optimization, and full MLOps orchestration for scaling this exact workflow to 50 million records.

All methodology, code structure, and modeling decisions in this report are fully reproducible and documented for reuse in future retraining or scaling efforts.